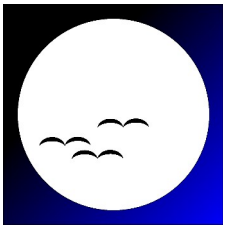


The QMdbSim2 Project

Basic User Documentation

Document History

2026.07.01 – Initial documentation. Caveat emptor.

Prerequisites

If you haven't already done so, please start with the `QMdbSim2_Overview.pdf`.

I assume that you have experience using Microchip MPLabX or similar tools to create micro-controller device code.

Parts of this project use Java. Pre-compiled JAR files are provided so you don't need to know anything about Java unless you wish to modify/compile the simulator interface code. If so, see the Device Developer and API Developer documentation.

Overview

This document details how to get set up to run `QMdbSim2` simulations as a Basic User. As a reminder, Basic Users:

- Create top-level schematics using device-specific symbols, schematics, and DLLs created by Device Developers.
- Create/modify Device Programs (`*.elf`/`*.hex` files) that run in the QSpice simulation.

We will be using example files from the `Basic_Demo` folder in the project repository. The example implements Microchip `PIC16F15213` and `PIC16F15214` devices (collectively `PIC16F1521x`) in a pre-compiled DLL. It also uses the pre-compiled `QMdbCS.jar` common to all `QMdbSim2` simulations. The sources and MPLabX project files are included for the Device Program.

Third-Party Tools

As a Basic User, you need to have Microchip's MPLabX IDE and a 32-bit version of a Java runtime environment (JRE) installed. While a bit annoying, *this a one-time setup*.

MPLabX IDE

I'm assuming that you already use MPLabX to develop programs for Microchip devices. The MPLabX installation contains the Microchip software simulator files used by QMdbSim2 so there's nothing extra to install.

However, you'll need at least MPLabX v6.30 for QMdbSim2 projects. Newer versions may work. Older versions of the software simulator have bugs.

Java JRE

As a Basic User, you don't really need to know anything about Java. But you'll need an installed 32-bit version of Java 8 JRE or better.

I really didn't want to say anything beyond that. However, it appears that 32-bit Java is being phased out and harder to find. And, for Basic User functions, I want to keep it simple so here's my recommendation:

Get the Eclipse Adoptium Java 8 JRE available here: Adoptium -- Download Temurin JDK. In the "Filter Releases" section near the top of the page, select "x86" in the Architecture filter. This will show only "Windows x32" with JDK and JRE options. Select JRE for the minimum install.

To the right, it will warn that this is an "Unsupported Platform." Click on the "Show downloads anyway" button and choose MSI or ZIP downloads.

- MSI will run a full installer and put the JRE in `C:\Program Files (x86)` and offer to set the Java installation location in the PATH variable.
- ZIP if you don't want the full install integration. You can unzip it into any folder and it doesn't alter the system configuration.

ZIP is probably better since it doesn't interfere with Java versions already installed and, therefore, shouldn't break anything. Either will work. Make a note of where the JRE is installed for later.

Note: As mentioned in the Overview documentation, Java 17 seems to run much slower than Java 8. I recommend Java 8 JRE for now.

The Demonstration Files

First Test

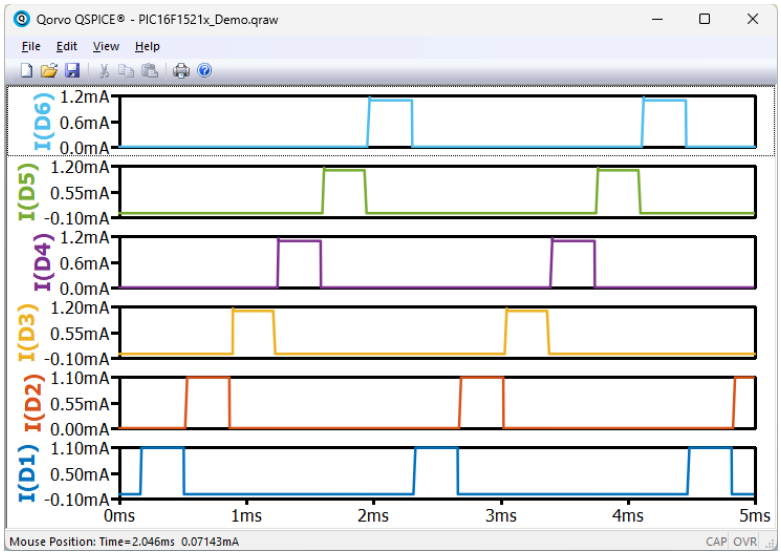
With the MPLabX and Java tools installed, we're almost ready to run `QMdbSim2` projects. Copy the `Basic_Demo` folder from the repository to your machine. You'll find the following files:

```
Basic_Demo
│   PIC16F1521x_Demo.qsch        Top-level schematic
│   PIC16F1521x.qsch             Device Schematic
│   PIC16F1521x.qsym             Device Symbol
│   Test_PIC16F1521x.elf         Device Program
│   GPIO_IMPL.qsch               GPIO pin implementation
│   PIC16F1521x.dll              Device DLL
│   QMdbCfg.ini                  QMdbSim2 configuration file
│
├───MPLabX_Source
│       PIC16F15213_Cplex.zip    Device Program sources
│
└───QMdbCS
        QMdbCS.jar               QMdbSim2 interface to Microchip simulator back-end
```

We need to make one change before we can run a simulation: Open `QMdbCfg.ini` with an ASCII text editor (e.g., `NotePad`). Edit the `MplabxRoot=…` and `JdkRoot=…` entries to point to MPLabX and Java JRE installed above. Note that there is also a `QmdbcsJar=.\QMdbCS\QMdbCS.jar` entry which uses a relative path to point the `QMdbCS.jar` file in the `Basic_Demo` files.

If all has gone to plan, you should now be able to run the example simulation. The example demonstrates using the PIC processor to drive 6 LEDs using 3 pins using a technique called "Charlie-Plexing." (Feel free to Google that.)

Open `PIC16F1521x_Demo.qsch` in QSpice and run the simulation. The simulation will take a bit to run so be patient. If all has gone to plan, you should see a Plot Window like the below:

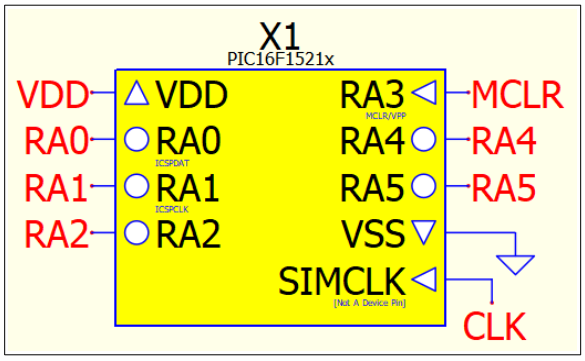


Note: Simulation execution times vary a bit wildly between runs. On my rather average laptop, the first simulation might take 30 seconds or more. A second run is much faster presumably due to JIT and/or Windows caching. Most of the simulation time is burned with the back-end simulator initialization; the time per simulation instruction step is something like 1.5ms / instruction (Java 17) or 0.6ms / instruction (Java 8). Your mileage will vary.

If the simulation run fails, double-check all of the configuration stuff above.

The Top-Level Schematic Device Symbol

Let's start with the schematic to see the project features. The Device Symbol looks like the image to the right.



The pin names should match the Microchip datasheet for the device except for the `SIMCLK` pin (discussed below).

The small text below the pin names is informational; the Device Developer is encouraged to use these to document alternate pin functionality as found in the

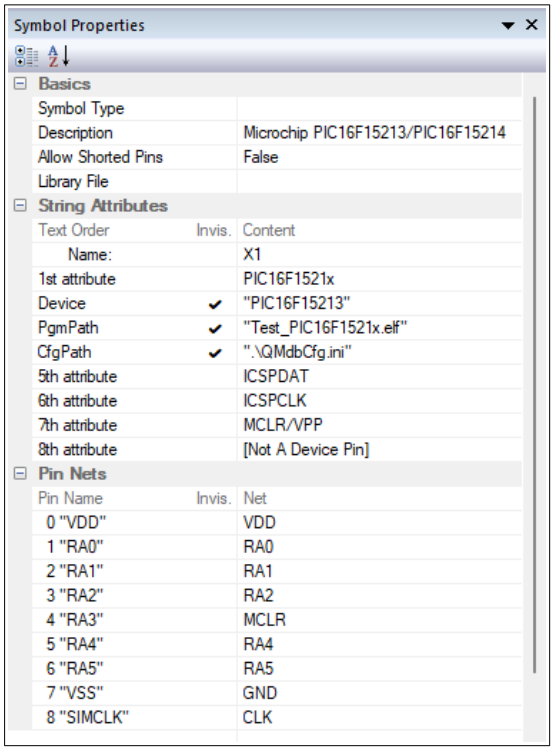
datasheet for quick reference. (Note that the presence of this text does not mean that the alternate functionality is actually present in the simulator.)

The graphical symbols beside the pins give information about the port direction. In the above, `VDD` is a voltage supply pin. `VSS` is a ground pin. `RA3` is an input-only pin (points into the center of the symbol). The other `RAx` pins are bi-directional (GPIO) pins. Not shown is an output-only pin which would point away from the center of the symbol.

That leaves the `SIMCLK` pin. This should be driven with a pulse source at the device's oscillator frequency divided by the number of ticks per instruction. See Device Programs below.

The Symbol Properties

Open the Symbol Properties for the Device Symbol. There are several important things to note.



Symbol Type

The Symbol Type is blank. That means that it loads a schematic named by the 1st String Attribute.

Description

The Description is purely informational and provided by the Device Developer. You can change it without consequence.

1st String Attribute

The 1st String Attribute names the Device Schematic which, in turn, loads the Device DLL to drive the back-end simulator. This schematic and the Device DLL are provided by Device Developers and not intended for modification by Basic Users.

Device String Attribute

The Device String Attribute contains a drop-down box to select the hardware device to be emulated. In this case, the `PIC16F1521x` Device Symbol supports both `PIC16F15213` & `PIC16F15214` devices so you can choose which one in this drop-down list.

PgmPath String Attribute

The `PgmPath` String Attribute is the path/filename of the compiled Device Program (`*.hex`/`*.elf`). Basic Users create whatever code is useful for their top-level schematic circuit, compile it, and provide the path here. Surround the path with double-quotes.

Note: What happens if you compile the Device Program for one device and select a different device in the Device String Attribute? I can't say for sure. The demonstration program was intended for

PIC16F15213 but doesn't fail if PIC16F51214 is selected. The chips are identical other than the 214 has twice as much memory. If the Device Program was compiled for a 214 the 213 was selected, I would expect the simulation to fail during initialization. But I've not tested this.

CfgPath String Attribute

The `CfgPath` String Attribute gives the path to the `QMdbCfg.ini` file to use for this simulation. In the `PIC16F15213_Demo.qsch` Device Symbol, this is set to "`.\QMdbCfg.ini`" which, of course, picks up the `QMdbCfg.ini` from the QSpice working directory. The `*.ini` in turn specifies that the `QMdbCS.jar` is in the `QMdbCS` sub-folder of the working directory.

Tip: Rather than copy the `*.jar` and `*.ini` files for each `QMdbSim2` schematic project, we can relocate these files to fixed locations. You could do this several ways but, for simplicity, put both in a single folder outside of the schematic project. Edit the `*.ini` to point to that folder/path. And, in top-level schematics, change the Device Symbol `CfgPath` to point to that location.

Basic User Development

Now that you have successfully configured the tools and run the example, future projects are much easier. Here are the basic steps:

- Obtain device-specific files for your target device (e.g., `PIC16F1521x.qsym`, `PIC16F1521x.qsch`, and `PIC16F1521x.dll`).
- Drag/drop the Device Symbol onto a new schematic.
- Add your top-level circuitry. Include a `SIMCLK` clock source that ticks at the device *instruction frequency*. (See SIMCLK/Oscillators below.)
- Write your device program code and compile to binary (`*.hex`/`*.elf`).
- Edit the device symbol properties (`Device`, `PgmPath`, `CfgPath`).
- Run your simulation.

Device Programs

Here are a few tricks & tips or limitations relevant to writing your Device Program code.

Peripheral Devices

Before you begin designing a project, make sure that any "peripherals" that you intend to use are supported by the Microchip simulator. For example, the datasheet may say that `PWM`, `ADC`, `SPI`, `I²C`, and/or `UART` peripherals are included in the device. However, the Microchip simulator may not support the peripherals.

See Microchip section in Resources below for a link to the Simulator Peripheral Support By Device.

SIMCLK/Oscillators

As a practical matter, the QSpice simulation must drive the software simulator to step through Device Program instructions. With each step, the `QMdbSim2` framework sets simulator pin states from QSpice input pins, advances the software simulation by one instruction cycle, and copies pins states from the software simulator back to QSpice output pins.

While a physical device may have internal oscillators and may even be able to switch internal oscillator speeds under device program control, the `QMdbSim2` framework doesn't currently support this. Set the `SIMCLK` to a frequency that matches your expected normal operating frequency. If your Device Program switches frequencies dynamically, well, that is not propagated back to QSpice so the simulation simply won't match real-world behavior.

Electrical Behavior

`QMdbSim2` devices are not intended to accurately simulate physical device behavior. They are implemented as QSpice DLL blocks. Input pins (ports) have infinite resistance. Output pins (ports) are *voltage sources* with fixed internal resistance.

This has consequences: If a physical device pin has dynamically configurable internal weak pull-up resistors, that means that the pin is connected to an open-collector/open-drain transistor. Again, the `QMdbSim2` device output pins are voltage sources so, if you want to present the pin to your circuit as OC/OD, you'll need to add a transistor to the simulated device pin. Further, dynamically switching internal pull-up resistors isn't supported at all so enable them in your Device Program from the start and add the pull-up resistor to the schematic.

Supply Voltage

This is related to the Peripheral Devices and Electrical Behavior topics above. The Microchip software simulator does not appear to allow changing the supply voltage after the simulation starts. That is, we can't change the supply voltage as seen by the simulator during the simulation. The consequence is that supply-voltage-dependent functionality (such as Brown-Out Detection) probably does not work. Further investigation is needed….

Multiple Instances/Stepped Simulations

I have not yet tested to ensure that multiple Device Symbol instances work properly. I also have not yet tested simulations with STEP directives. These may or may not work so proceed with caution.

For more on these topics, see the Device Developer documentation.

Resources

My GitHub QSpice Repository

https://github.com/robdunn4/QSpice

Microchip

MPLabX IDE (free)

Microchip MPLabX SDK / MDBCore API

Microchip Debugger (MDB) User's Guide (one)

Microchip Debugger (MDB)User’s Guide (another one)

Simulator Peripheral Support By Device

Microsoft

Visual Studio Downloads (2026 Community Edition is free)

Java JDK/JRE

Eclipse Adoptium Temurin (free open source Java JDK/JRE)

Conclusion

I hope that you find this project to be useful or – at least – interesting. Please let me know if you find problems or suggestions for improving the code or this documentation.

You can find me on the QSpice Forums as @RDunn.

--robert